

Distributed Training

 llamafactory.readthedocs.io/en/latest/advanced/distributed.html

LLaMA-Factory supports single-node multi-card and Multi-node Multi-card distributed training. It also supports three distributed engines: [DDP](#), [DeepSpeed](#), and [FSDP](#).

[DDP](#) (DistributedDataParallel) achieves training acceleration through model parallelism and data parallelism. Programs using DDP need to spawn multiple processes and create a DDP instance for each process, which synchronize through the `torch.distributed` library.

[DeepSpeed](#) is a distributed training engine developed by Microsoft, offering optimization techniques such as ZeRO (Zero Redundancy Optimizer), offload, Sparse Attention, 1 bit Adam, and pipeline parallelism. You can choose to use it based on task requirements and device availability.

[FSDP](#) processes more and larger models through Fully Sharded Data Parallel technology. In DDP, each GPU maintains a complete copy of model parameters and optimizer parameters. FSDP shards model parameters, gradients, and optimizer parameters, allowing each GPU to only retain a portion of these parameters. Beyond parallelism techniques, FSDP also supports offloading model parameters to CPU, further reducing memory requirements.

[FSDP2](#), while integrating the basic functionality of FSDP1, abandons FSDP1's approach of flattening and concatenating parameters and instead implements per-parameter sharding based on DTensor. This architectural upgrade, while fully preserving the original model structure, significantly improves the overlap efficiency of computation and communication, thereby enhancing training performance.

Engine	Data Sharding	Model Sharding	Optimizer Sharding	Parameter Offload
DDP	Supported	Not Supported	Not Supported	Not Supported
DeepSpeed	Supported	Supported	Supported	Supported
FSDP	Supported	Supported	Supported	Supported
FSDP2	Supported	Supported	Supported	Supported

NativeDDP

NativeDDP is a distributed training method provided by PyTorch. You can start training with the following command:

Single-node Multi-GPU

llamafactory-cli

You can use llamafactory-cli to start the NativeDDP engine.

```
FORCE_TORCHRUN=1 llamafactory-cli train examples/train_full/llama3_full_sft_ds3.yaml
```

If `CUDA_VISIBLE_DEVICES` is not specified, all GPUs will be used by default. If you need to specify GPUs, for example GPUs 0 and 1, you can use:

```
FORCE_TORCHRUN=1 CUDA_VISIBLE_DEVICES=0,1 llamafactory-cli train config/config1.yaml
```

torchrun

You can also use the `torchrun` command to start the NativeDDP engine for single-node multi-card training. Here is an example:

```
torchrun --standalone --nnodes=1 --nproc-per-node=8 src/train.py \
--stage sft \
--model_name_or_path meta-llama/Meta-Llama-3-8B-Instruct \
--do_train \
--dataset alpaca_en_demo \
--template llama3 \
--finetuning_type lora \
--output_dir saves/llama3-8b/lora/ \
--overwrite_cache \
--per_device_train_batch_size 1 \
--gradient_accumulation_steps 8 \
--lr_scheduler_type cosine \
--logging_steps 100 \
--save_steps 500 \
--learning_rate 1e-4 \
--num_train_epochs 2.0 \
--plot_loss \
--bf16
```

accelerate

You can also use the `accelerate` command to start single-node multi-card training.

First run the following command, answer a series of questions based on your needs, and generate a configuration file:

Here is an example configuration file:

```
# accelerate_singleNode_config.yaml
compute_environment: LOCAL_MACHINE
debug: true
distributed_type: MULTI_GPU
downcast_bf16: 'no'
enable_cpu_affinity: false
gpu_ids: all
machine_rank: 0
main_training_function: main
mixed_precision: fp16
num_machines: 1
num_processes: 8
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

You can start training by running the following command:

```
accelerate launch \
--config_file accelerate_singleNode_config.yaml \
src/train.py training_config.yaml
```

Multi-node Multi-card

llamafactory-cli

```
FORCE_TORCHRUN=1 NNODES=2 NODE_RANK=0 MASTER_ADDR=192.168.0.1 MASTER_PORT=29500 \
llamafactory-cli train examples/train_lora/llama3_lora_sft.yaml
```

```
FORCE_TORCHRUN=1 NNODES=2 NODE_RANK=1 MASTER_ADDR=192.168.0.1 MASTER_PORT=29500 \
llamafactory-cli train examples/train_lora/llama3_lora_sft.yaml
```

Variable Name	Description
FORCE_TORCHRUN	Whether to force the use of torchrun
NNODES	Number of nodes
NODE_RANK	Rank of each node.
MASTER_ADDR	Address of the master node.

Variable Name	Description
MASTER_PORT	Port of the master node.

torchrun

You can also use the **torchrun** command to start the NativeDDP engine for Multi-node Multi-card training.

```
torchrun --master_port 29500 --nproc_per_node=8 --nnodes=2 --node_rank=0 \
--master_addr=192.168.0.1 train.py
torchrun --master_port 29500 --nproc_per_node=8 --nnodes=2 --node_rank=1 \
--master_addr=192.168.0.1 train.py
```

accelerate

You can also use the **accelerate** command to start Multi-node Multi-card training.

First run the following command, answer a series of questions based on your needs, and generate a configuration file:

Here is an example configuration file:

```
# accelerate_multiNode_config.yaml
compute_environment: LOCAL_MACHINE
debug: true
distributed_type: MULTI_GPU
downcast_bf16: 'no'
enable_cpu_affinity: false
gpu_ids: all
machine_rank: 0
main_process_ip: '192.168.0.1'
main_process_port: 29500
main_training_function: main
mixed_precision: fp16
num_machines: 2
num_processes: 16
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

You can start training by running the following command:

```
accelerate launch \
--config_file accelerate_multiNode_config.yaml \
train.py llm_config.yaml
```

DeepSpeed

DeepSpeed is an open-source deep learning optimization library developed by Microsoft, aimed at improving the efficiency and speed of large model training. Before using DeepSpeed, you need to first estimate the memory size of the training task, then select an appropriate ZeRO stage based on task requirements and resource availability.

- ZeRO-1: Only shards optimizer parameters, each GPU has a complete copy of model parameters and gradients.
- ZeRO-2: Shards optimizer parameters and gradients, each GPU has a complete copy of model parameters.
- ZeRO-3: Shards optimizer parameters, gradients, and model parameters.

Simply summarize: from ZeRO-1 to ZeRO-3, the higher the stage number, the smaller the memory requirement, but training speed also decreases accordingly. Additionally, setting the `offload_param=cpu` parameter will significantly reduce memory requirements, but will greatly slow down training speed. Therefore, if you have sufficient memory, you should use ZeRO-1 and ensure `offload_param=None`.

LLaMA-Factory provides examples of DeepSpeed configuration files using different stages. Including:

- [ZeRO-0](#) (disabled)
- [ZeRO-2](#)
- [ZeRO-2+offload](#)
- [ZeRO-3](#)
- [ZeRO-3+offload](#)
- [ZeRO-1/2+AutoTP](#)

Single-node Multi-GPU

llamafactory-cli

You can use llamafactory-cli to start the DeepSpeed engine for single-node multi-card training.

```
FORCE_TORCHRUN=1 llamafactory-cli train examples/train_full/llama3_full_sft_ds3.yaml
```

To start the DeepSpeed engine, the `deepspeed` parameter in the configuration file specifies the path to the DeepSpeed configuration file:

```
...
deepspeed: examples/deepspeed/ds_z3_config.json
...
```

deepspeed

You can also use the **deepspeed** command to start the DeepSpeed engine for single-node multi-card training.

```
deepspeed --include localhost:1 your_program.py <normal cl args> --deepspeed
ds_config.json
```

Here is an example:

```
deepspeed --num_gpus 8 src/train.py \
--deepspeed examples/deepspeed/ds_z3_config.json \
--stage sft \
--model_name_or_path meta-llama/Meta-Llama-3-8B-Instruct \
--do_train \
--dataset alpaca_en \
--template llama3 \
--finetuning_type full \
--output_dir saves/llama3-8b/lora/full \
--overwrite_cache \
--per_device_train_batch_size 1 \
--gradient_accumulation_steps 8 \
--lr_scheduler_type cosine \
--logging_steps 10 \
--save_steps 500 \
--learning_rate 1e-4 \
--num_train_epochs 2.0 \
--plot_loss \
--bf16
```

Note

When starting the DeepSpeed engine using the **deepspeed** command, you cannot use **CUDA_VISIBLE_DEVICES** to specify GPUs. Instead, you need to:

```
deepspeed --include localhost:1 your_program.py <normal cl args> --deepspeed
ds_config.json
```

--include localhost:1 means only use gpu1 on this node.

Multi-node Multi-card

LLaMA-Factory supports multi-node multi-card training using DeepSpeed. You can start it with the following command:

```
FORCE_TORCHRUN=1 NNODES=2 NODE_RANK=0 MASTER_ADDR=192.168.0.1 MASTER_PORT=29500  
llamafactory-cli train examples/train_lora/llama3_lora_sft_ds3.yaml  
FORCE_TORCHRUN=1 NNODES=2 NODE_RANK=1 MASTER_ADDR=192.168.0.1 MASTER_PORT=29500  
llamafactory-cli train examples/train_lora/llama3_lora_sft_ds3.yaml
```

deepspeed

You can also use the **deepspeed** command to start multi-node multi-card training.

```
deepspeed --num_gpus 8 --num_nodes 2 --hostfile hostfile --master_addr hostname1 --  
master_port=9901 \  
your_program.py <normal cl args> --deepspeed ds_config.json
```

Note

- **About hostfile:**

Each line of the hostfile specifies a node, with the format **<hostname> slots=<num_slots>**, where **<hostname>** is the hostname of the node, and **<num_slots>** is the number of GPUs on that node. Here is an example: .. code-block:

```
worker-1 slots=4  
worker-2 slots=4
```

Please visit <https://www.deepspeed.ai/getting-started/> to learn more.

- If the **hostfile** variable is not specified, DeepSpeed will search for the **/job/hostfile** file. If still not found, then DeepSpeed will use all available GPUs on the local machine.

accelerate

You can also use the **accelerate** command to start the DeepSpeed engine. First generate the DeepSpeed configuration file with the following command:

Here is an example configuration file:

```
# deepspeed_config.yaml
compute_environment: LOCAL_MACHINE
debug: false
deepspeed_config:
  deepspeed_multinode_launcher: standard
  gradient_accumulation_steps: 8
  offload_optimizer_device: none
  offload_param_device: none
  zero3_init_flag: false
  zero_stage: 3
distributed_type: DEEPSPEED
downcast_bf16: 'no'
enable_cpu_affinity: false
machine_rank: 0
main_process_ip: '192.168.0.1'
main_process_port: 29500
main_training_function: main
mixed_precision: fp16
num_machines: 2
num_processes: 16
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

Then, you can start training with the following command:

```
accelerate launch \
--config_file deepspeed_config.yaml \
train.py llm_config.yaml
```

DeepSpeed Configuration File

ZeRO-0

```
### ds_z0_config.json
{
  "train_batch_size": "auto",
  "train_micro_batch_size_per_gpu": "auto",
  "gradient_accumulation_steps": "auto",
  "gradient_clipping": "auto",
  "zero_allow_untested_optimizer": true,
  "fp16": {
    "enabled": "auto",
    "loss_scale": 0,
    "loss_scale_window": 1000,
    "initial_scale_power": 16,
    "hysteresis": 2,
    "min_loss_scale": 1
  },
  "bf16": {
    "enabled": "auto"
  },
  "zero_optimization": {
    "stage": 0,
    "allgather_partitions": true,
    "allgather_bucket_size": 5e8,
    "overlap_comm": true,
    "reduce_scatter": true,
    "reduce_bucket_size": 5e8,
    "contiguous_gradients": true,
    "round_robin_gradients": true
  }
}
```

ZeRO-2

Simply modify the `stage` parameter in `zero_optimization` based on ZeRO-0.

```
### ds_z2_config.json
{
  ...
  "zero_optimization": {
    "stage": 2,
    ...
  }
}
```

ZeRO-2+offload

Simply add the `offload_optimizer` parameter in `zero_optimization` based on ZeRO-0.

```

### ds_z2_offload_config.json
{
    ...
    "zero_optimization": {
        "stage": 2,
        "offload_optimizer": {
            "device": "cpu",
            "pin_memory": true
        },
        ...
    }
}

```

ZeRO-3

Simply modify the parameters in `zero_optimization` based on ZeRO-0.

```

### ds_z3_config.json
{
    ...
    "zero_optimization": {
        "stage": 3,
        "overlap_comm": true,
        "contiguous_gradients": true,
        "sub_group_size": 1e9,
        "reduce_bucket_size": "auto",
        "stage3_prefetch_bucket_size": "auto",
        "stage3_param_persistence_threshold": "auto",
        "stage3_max_live_parameters": 1e9,
        "stage3_max_reuse_distance": 1e9,
        "stage3_gather_16bit_weights_on_model_save": true
    }
}

```

ZeRO-3+offload

Simply add the `offload_optimizer` and `offload_param` parameters in `zero_optimization` based on ZeRO-3.

```

### ds_z3_offload_config.json
{
  ...
  "zero_optimization": {
    "stage": 3,
    "offload_optimizer": {
      "device": "cpu",
      "pin_memory": true
    },
    "offload_param": {
      "device": "cpu",
      "pin_memory": true
    },
    ...
  }
}

```

ZeRO-1/2+AutoTP

Tensor Parallelism (TP) is an important memory optimization technique for training large-scale models. Previously, model scaling in Hugging Face Trainer mainly relied on sharded data parallelism via ZeRO/FSDP. While ZeRO-3 provides excellent memory efficiency, it introduces significant communication overhead. ZeRO-1/2 have lower communication costs, but are limited by memory capacity when training very large models.

To address this, DeepSpeed introduces native Automatic Tensor Parallel (AutoTP) training with official support for Hugging Face Transformers. AutoTP can be combined with ZeRO to achieve a better balance between performance and memory efficiency during training, providing the following benefits:

- Model scaling with lower communication overhead than FSDP / ZeRO-3 (e.g., AutoTP + ZeRO-1 can achieve memory savings comparable to ZeRO-3).
- Support for larger batch sizes to improve training throughput.
- Support for longer context lengths, enabling new application scenarios.

AutoTP training currently supports **ZeRO-1** and **ZeRO-2**. This feature is available in **DeepSpeed version 0.16.4 and later**.

The configuration parameters are as follows:

```

### ds2_autotp.json
{
  "ZeRO_optimization": {
    "stage": 2,
    ...
  },
  "tensor_parallel": {
    "autotp_size": 4
  },
}

```

Model support is currently limited. Please refer to the [AutoTP supported model list](#) for details.

FSDP

PyTorch's Fully Sharded Data Parallel technique [FSDP](#) allows us to handle more and larger models. LLaMA-Factory supports distributed training using the FSDP engine.

Different values of the FSDP parameter [ShardingStrategy](#) determine how the model is sharded:

- [FULL_SHARD](#): Shards model parameters, gradients, and optimizer states across different GPUs, similar to ZeRO-3.
- [SHARD_GRAD_OP](#): Shards gradients and optimizer states across different GPUs, while each GPU still maintains a complete copy of model parameters. Similar to ZeRO-2.
- [NO_SHARD](#): Does not shard any parameters. Similar to ZeRO-0.

Single-node Multi-GPU

llamafactory-cli

You just need to modify [examples/accelerate/fsdp_config.yaml](#) and [examples/extras/fsdp_qlora/llama3_lora_sft.yaml](#) as needed, then run the following command to start FSDP+QLoRA fine-tuning:

```
bash examples/extras/fsdp_qlora/train.sh
```

accelerate

Additionally, you can also use accelerate to start the FSDP engine. **The number of nodes and GPUs can be specified through num_machines and num_processes.** For this, Huggingface provides convenient configuration functionality. Just run:

After answering a series of questions according to the prompts, we can generate the configuration file required by FSDP.

Of course, you can also configure [fsdp_config.yaml](#) yourself according to your needs.

```

### /examples/accelerate/fsdp_config.yaml
compute_environment: LOCAL_MACHINE
debug: false
distributed_type: FSDP
downcast_bf16: 'no'
fsdp_config:
  fsdp_auto_wrap_policy: TRANSFORMER_BASED_WRAP
  fsdp_backward_prefetch: BACKWARD_PRE
  fsdp_forward_prefetch: false
  fsdp_cpu_ram_efficient_loading: true
  fsdp_offload_params: true # offload may affect training speed
  fsdp_sharding_strategy: FULL_SHARD
  fsdp_state_dict_type: FULL_STATE_DICT
  fsdp_sync_module_states: true
  fsdp_use_orig_params: true
machine_rank: 0
main_training_function: main
mixed_precision: fp16 # or bf16
num_machines: 1 # the number of nodes
num_processes: 2 # the number of GPUs in all nodes
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false

```

Note

Please ensure that `num_processes` matches the total number of GPUs actually used

Then, you can start training with the following command:

```

accelerate launch \
--config_file fsdp_config.yaml \
src/train.py llm_config.yaml

```

Warning

Do not use GPTQ/AWQ models with FSDP+QLoRA

Multi-node Multi-card

accelerate

You can use *accelerate config* to answer a series of questions and generate the configuration file required for multi-node FSDP.

Of course, you can also configure `fsdp_config.yaml` yourself according to your needs.

```
#examples/accelerate/fsdp_config_multiple_nodes.yaml
compute_environment: LOCAL_MACHINE
debug: false
distributed_type: FSDP
downcast_bf16: 'no'
fsdp_config:
  fsdp_auto_wrap_policy: TRANSFORMER_BASED_WRAP
  fsdp_backward_prefetch: BACKWARD_PRE
  fsdp_forward_prefetch: false
  fsdp_cpu_ram_efficient_loading: true
  fsdp_offload_params: false
  fsdp_sharding_strategy: FULL_SHARD
  fsdp_state_dict_type: FULL_STATE_DICT
  fsdp_sync_module_states: true
  fsdp_use_orig_params: true
machine_rank: 0
main_training_function: main
mixed_precision: bf16 # or fp16
main_process_ip: 192.168.0.1
main_process_port: 29500
num_machines: 2
num_processes: 16
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

In this YAML file, the key parameters you need to configure are the following four:

- `num_machines`: Number of nodes (machines).
- `num_processes`: Total number of GPUs across all nodes, i.e., `num_machines * num_processes_per_machine`.
- `main_process_ip`: IP address of the node where the main process runs; ensure all nodes use the same IP.
- `main_process_port`: Port of the main process; ensure all nodes use the same port.
- `machine_rank`: Index of the current node (machine), starting from 0; the main node (`main_process_ip`) must have `machine_rank` 0.

After the configuration is complete, run the following command on all machines to start multi-node multi-card FSDP training:

```
accelerate launch \
--config_file fsdp_config_multiple_nodes.yaml \
train.py llm_config.yaml
```

FSDP2

Currently, LLaMA-Factory uses FSDP2 for distributed training based on Accelerate integration. The main difference from FSDP lies in parameter configuration. You can run single-node multi-card and multi-node multi-card in the same way as FSDP. We also provide general input parameter configurations:

```
#examples/accelerate/fsdp2_config.yaml
compute_environment: LOCAL_MACHINE
debug: false
distributed_type: FSDP
downcast_bf16: 'no'
fsdp_config:
  fsdp_auto_wrap_policy: TRANSFORMER_BASED_WRAP
  fsdp_cpu_ram_efficient_loading: true
  fsdp_offload_params: false
  fsdp_reshard_after_forward: true
  fsdp_state_dict_type: FULL_STATE_DICT
  fsdp_version: 2
machine_rank: 0
main_training_function: main
mixed_precision: bf16 # or fp16
num_machines: 1 # the number of nodes
num_processes: 2 # the number of GPUs in all nodes
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

You can quickly start the training script with the following command:

```
accelerate launch \
--config_file fsdp2_config.yaml \
train.py llm_config.yaml
```

For more differences in input parameter configurations, refer to [Accelerate FSDP1 vs FSDP2](#).

Ray

Currently, LlamaFactory also supports enabling Ray for distributed training by setting the environment variable `USE_RAY=1`. You can refer to the [Ray official documentation](#) for more information.

Single-node Multi-GPU

You can specify the number of GPUs to use by setting the `num_workers` parameter.

NativeDDP, DeepSpeed

You can use the llamafactory-cli command to start the NativeDDP engine and DeepSpeed engine.

```
USE_RAY=1 llamafactory-cli train training_config.yaml
```

FSDP

You can use the accelerate command to start the FSDP engine. When starting with accelerate, you need to set the `num_processes` parameter in the `fsdp_config.yaml` file to 1.

```
USE_RAY=1 accelerate launch \  
--config_file fsdp_config.yaml \  
src/train.py training_config.yaml
```

Multi-node Multi-card

When using Ray for multi-machine multi-GPU training, you first need to run the following commands on each node separately.

Master node:

```
ray start --head --port=6379
```

Worker node:

```
ray start --address='master node IP:6379'
```

Then, you can use the same commands as single-machine training on the master node to start training, or use the `ray job submit` command to submit training tasks.

NativeDDP, DeepSpeed

```
USE_RAY=1 llamafactory-cli train training_config.yaml
```

Submit training tasks using the `ray job submit` command.

```
RAY_API_SERVER_ADDRESS='http://dashboard-host:dashboard-port \  
ray job submit -- llamafactory-cli train training_config.yaml
```

FSDP

Similarly, you need to set the `num_processes` parameter in the `fsdp_config.yaml` file to 1, and you do not need to set any multi-machine related parameters.

```
USE_RAY=1 accelerate launch \  
--config_file fsdp_config.yaml \  
src/train.py training_config.yaml
```

Submit training tasks using the `ray job submit` command.


```
RAY_API_SERVER_ADDRESS='http://dashboard-host:dashboard-port \'  
ray job submit -- \  
USE_RAY=1 accelerate launch \  
--config_file fsdp_config.yaml \  
src/train.py training_config.yaml
```

Copyright © 2024, LlamaFactory team.